Kế hoạch thực hiện đề tài 26128

1 Mục tiêu và quyết định khởi động

Asynchronous virtio-blk and I/O schedulers trên xv6-riscv

Nhóm cần xây dựng một đường xử lý I/O có hàng đợi, cho phép nhiều request đang chờ thiết bị đồng thời, có ba chính sách FCFS, SCAN và deadline, hỗ trợ gộp request và API đọc bất đồng bộ. Kết quả phải được chứng minh bằng kiểm thử tự động, benchmark và demo chuyển chính sách khi hệ thống đang chạy. Đây là đề tài I/O mức khó ★★★ trong catalogue. [S1]

Trong tuần đầu, ưu tiên thống nhất phạm vi, làm cho cả ba máy chạy được xv6, đo baseline, đọc luồng I/O và chốt thiết kế sơ bộ. Kết thúc tuần cần có bằng chứng kỹ thuật và backlog thực tế; chưa cần hoàn thành cả ba scheduler. Từ tuần 2 triển khai request lifecycle và FCFS trước, sau đó mới tích hợp SCAN, deadline và merging.

Thông tin nhóm và giả định lập lịch

Thành viên theo catalogue	MSSV	Ký hiệu trong tài liệu
Nguyễn Văn Hoàng	20233413	Hoàng
Phạm Qúy Lợi	20233501	Lợi
Phạm Trung Hiếu	20233402	Hiếu

Tên được giữ theo catalogue; nhóm kiểm tra lại chính tả trước khi nộp. Phân công dưới đây là đề xuất để nhóm chốt trong buổi kickoff, chưa phản ánh công việc đã làm.

Theo thông tin nhóm cung cấp, đề tài đã chốt tuần trước, hiện là tuần làm việc đầu tiên và lịch nộp ở tuần 10. Lịch kế hoạch dùng thứ Hai 19/10/2026 làm ngày đầu tuần 1; tuần 10 là 21–27/12/2026. Các khoảng ngày là khung làm việc, chưa phải hạn nộp cụ thể trong ngày.

Spec ghi mốc theo tuần học phần: tuần 1 đăng ký, tuần 3 proposal, tuần 5 midterm, tuần 8 code freeze, tuần 9 final demo và báo cáo. Nhóm cần đối chiếu tuần làm việc với tuần học phần khi giảng viên xác nhận. Trong lúc đó, kế hoạch giữ các mốc sớm tuần 3/5/8/9, dùng tuần 10 cho nộp hồ sơ theo lịch nhóm thông báo. [S2]

Cách dùng bộ tài liệu

Đọc báo cáo này để hiểu phạm vi và lộ trình. Dùng 02_Checklist_tuan_1.md để bắt đầu ngay; đưa các task trong 04_Backlog_va_phan_cong.md lên Jira; dùng 05_Ke_hoach_kiem_thu.md để thiết kế test từ sớm. Bản proposal 03 là dự thảo thiết kế để hoàn thiện ở tuần 3. Báo cáo tuần 1 phải điền kết quả thực tế theo tài liệu 06 và mẫu gốc, không lấy kế hoạch làm thành tích.

2 Những yêu cầu nhóm phải đáp ứng

Mã	Yêu cầu bắt buộc	Bằng chứng nghiệm thu dự kiến
R01	Nhiều virtio request outstanding	Trace submit/complete có ít nhất 2 request INFLIGHT cùng lúc; dữ liệu đúng
R02	FCFS	Test dispatch theo thứ tự đến với workload xác định
R03	SCAN	Trace đầu đọc logic đi đến biên rồi đảo chiều; phân biệt với LOOK
R04	Deadline	Test ưu tiên request đã hết hạn, chống bỏ đói theo quy tắc đã công bố
R05	Request merging	Request liền kề hợp lệ thành một lệnh thiết bị; checksum và completion từng request đúng
R06	Async read API	Submit trả trước completion; caller làm việc khác rồi wait/poll; dữ liệu đúng
R07	So sánh scheduler lúc chạy	Công cụ chuyển FCFS/SCAN/deadline và đồ thị throughput/latency cùng workload
R08	Driver và request queue	Mã kernel, cấu trúc request, trạng thái, khóa và xử lý tài nguyên
R09	Stress tối thiểu 30 phút	Log có thời lượng, seed, cấu hình CPU, checksum, tài nguyên và kết luận

R01–R09 được rút từ dòng đề tài 26128; cách chứng minh trong cột cuối là đề xuất của kế hoạch. Seek-time model và ổ đĩa thứ hai là stretch, chỉ làm sau khi phần bắt buộc ổn định. [S1]

Yêu cầu chung của học phần

Repository Git phải để riêng tư và chia sẻ với giảng viên. Cả ba thành viên cần có commit đáng kể trong kernel. Mã phải build bằng một lệnh; bộ test chạy bằng một lệnh và in PASS/FAIL từng ca. Cuối kỳ phải giữ stock usertests và grind hoạt động, trừ thay đổi ngữ nghĩa được giải thích. [S2]

Concurrency/stress phải chạy CPUS=1 và CPUS=4, lặp tối thiểu 10 lần. Cần kiểm tra free pages và free inodes trước/sau suite. Mỗi điểm benchmark có ít nhất 5 lần đo, trình bày trung bình và độ phân tán. Mục tiêu nội bộ là ít nhất 30 ca có assertion thực chất, không tăng số lượng bằng cách đổi tên cùng một ca. [S2]

Jira là lựa chọn mặc định; Trello hoặc GitHub Projects cần được chấp thuận. Figma/FigJam hoặc công cụ sơ đồ tương đương là bắt buộc. Từ tuần học phần 2, giảng viên phải đọc được repo, tracker và design. Báo cáo tiến độ nộp hằng tuần đến code freeze, có đóng góp cá nhân, đối chiếu TODO kỳ trước, bằng chứng và kế hoạch kỳ sau. [S2]

Điểm cần xác nhận ngay

Chốt tag/commit xv6 gốc; quy đổi tuần học phần và lịch làm việc; ngày giờ nộp tuần 10; lịch demo; API đọc bất đồng bộ có bắt buộc user syscall hay chấp nhận API kernel có chương trình minh họa; định nghĩa SCAN giảng viên mong đợi. Spec mô tả Functionality 30 + Testing 20 nhưng bảng rubric ghi Testing 5 và Measurement 5; bảng cộng được 100. Không tự suy ra trọng số thay giảng viên. [S2]

3 Kiến thức và môi trường cần chuẩn bị

Kiến thức tối thiểu cho cả ba thành viên

Mỗi người cần hiểu C con trỏ, struct, vòng đời bộ nhớ và error path; trap/interrupt; spinlock và sleeplock; điều kiện chờ và wakeup; block/sector; DMA descriptor và virtqueue; buffer cache và logging của xv6. Hoàng đọc sâu đường virtio và completion; Lợi đọc thuật toán hàng đợi và đồng bộ; Hiếu đọc buffer cache, logging và thiết kế phép đo. Cả nhóm vẫn review chéo để giải thích được toàn bộ đường I/O.

Bộ công cụ và cấu hình tái lập

Hạng mục	Chuẩn bị	Đầu ra cần lưu
Máy phát triển	Đề xuất WSL2 Ubuntu trên Windows hoặc Linux VM; dùng cùng một môi trường trong nhóm	Phiên bản OS và cách cài trong README
Build và chạy	Git, GNU make, compiler RISC-V GNU đúng prefix, QEMU RISC-V	Kết quả version, lệnh make qemu, cấu hình CPUS
Phân tích và đo	Python, script điều khiển QEMU, trình vẽ đồ thị	Lệnh chạy, seed và định dạng dữ liệu
Debug	GDB hỗ trợ RISC-V; serial log; addr2line khi cần	Hướng dẫn breakpoint submit/interrupt
Quản lý	Repo private, Jira, FigJam; quyền đọc của giảng viên	URL thực và kiểm tra quyền truy cập

Không lấy HEAD mới nhất thay cho tag giảng viên chỉ định. Ghi lại commit gốc và phiên bản công cụ trước khi sửa kernel. README upstream có hướng dẫn build; tên gói cài đặt phụ thuộc bản Linux đã chọn nên nhóm kiểm tra sau khi chốt môi trường. [S5]

Trình tự kiểm tra môi trường

1. Nhận tag/commit chính thức; clone vào workspace phát triển riêng, ghi git rev-parse HEAD.
2. Build sạch và boot bằng make qemu; ghi lại đầy đủ command line và ảnh/log shell.
3. Thực hiện đọc/ghi file đơn giản, chạy stock usertests và grind với giới hạn thời gian được ghi rõ.
4. Lặp kiểm tra baseline trên CPUS=1 và CPUS=4; lưu mã thoát, số PASS/FAIL hoặc thời lượng workload.
5. Dùng cùng commit và cấu hình để máy thứ hai tái lập được kết quả; máy thứ ba xác nhận boot/build.

Đây là các việc nhóm phải thực hiện. Bộ tài liệu hiện tại không khẳng định đã cài toolchain, chạy QEMU hay có test PASS. Với grind dạng chạy dài, ghi thời lượng và cách kết thúc, không tự ghi PASS chỉ vì chưa thấy panic.

Cấu trúc repository đề xuất

Giữ kernel/ và user/ theo xv6; bổ sung tests/, scripts/, docs/, results/ và README.md. Trong results/, mỗi lần chạy có metadata, serial log, dữ liệu thô và file tổng hợp. Gắn issue vào commit và PR; lưu ảnh FigJam trong docs/ cùng URL bản chỉnh sửa. T1 là bộ hồ sơ chuẩn bị, không phải repository mã kernel đã triển khai.

4 Hướng thiết kế kỹ thuật đề xuất

Tách các trách nhiệm trong đường I/O

Đường đồng bộ hiện hữu nên được giữ qua một wrapper: caller → submit request → wait completion → trả kết quả. Đường mới cho phép caller submit nhiều lần trước khi wait. Một hàng đợi phần mềm đứng trước virtqueue; scheduler chỉ chọn các request đang QUEUED. Interrupt nhận completion, cập nhật trạng thái và kích hoạt dispatch tiếp khi còn descriptor.

Luồng đề xuất: API đọc hoặc wrapper đồng bộ → request pool → pending queue → merge hợp lệ → FCFS/SCAN/deadline → virtqueue → thiết bị → interrupt → completion → caller thu hồi kết quả. Chi tiết được tách trong bản proposal và sơ đồ Mermaid.

Upstream virtio_disk_rw chờ completion theo từng caller, nhưng có theo dõi nhiều descriptor chain. Vì vậy cần đo số request thực tế cùng INFLIGHT ở baseline; không được mô tả stock xv6 là chỉ có đúng một request toàn hệ thống. Mã upstream hiện tại cũng có thể khác tag học phần về API sleep/wakeup. [S5]

Request và quyền sở hữu

Đề xuất request có id kèm generation, device, operation, sector bắt đầu, byte count, buffer/segment list, trạng thái, mã lỗi, thời điểm enqueue/dispatch/complete, deadline, nhóm ordering và thông tin các request con nếu merged. Pool có giới hạn, không cấp phát vô hạn dưới tải.

Trạng thái cơ bản: FREE → ALLOCATED → QUEUED → INFLIGHT → DONE hoặc ERROR → FREE. Với merging, request con thuộc nhóm cha; completion cha phải hoàn tất đúng một lần cho từng con. Chỉ merge request còn QUEUED. Không tái dùng handle hoặc bộ nhớ DMA trước khi thiết bị xong và caller đã thu hồi kết quả.

Submit là thao tác không chờ thiết bị sau khi đã chuẩn bị buffer; pool/queue đầy thì trả lỗi được định nghĩa. Wrapper đồng bộ có thể chờ tài nguyên theo cơ chế chờ của tag gốc. API đề xuất gồm aio_read_submit, aio_poll, aio_wait và aio_release; đây chưa phải tên syscall đã chốt.

Buffer cache và write ordering

Giai đoạn đầu giữ bread/bwrite đồng bộ và nguyên tắc ownership của buf. Không để interrupt thả sleeplock thuộc caller. Với async, buffer phải được giữ hợp lệ tới completion bằng ownership/refcount rõ ràng; mọi copy sang user space hoặc xử lý có thể sleep thực hiện trong process context. Không giữ một user pointer tự do cho DMA sau khi process thoát.

Để giảm rủi ro logging, chỉ reorder/merge các read độc lập trong bản đầu; write đi theo đường tương thích và đợi completion như trước. Đề xuất write fence: đợi các request trước fence xong, chạy write, hoàn tất write rồi mới dispatch nhóm sau. Việc mở rộng write merging chỉ thực hiện khi có mô hình phụ thuộc và test riêng. Cần hỏi giảng viên phạm vi merging được chấp nhận. [S6, S7]

5 Thuật toán và các điểm khó cần giải quyết

Ba chính sách trên cùng một queue

FCFS chọn request có thứ tự enqueue sớm nhất trong nhóm đủ điều kiện. Đây là đường chuẩn để kiểm tra lifecycle trước khi thêm tối ưu. Với SCAN, lưu vị trí đầu đọc logic và chiều quét; chọn sector tiếp theo theo chiều hiện tại, đi đến biên thiết bị rồi đảo chiều. Biên phải lấy từ capacity, không nhầm sector với block. Nếu quay đầu ngay ở request cuối cùng thì đó là LOOK, cần ghi tên đúng hoặc xin chấp thuận. Không tạo I/O giả tới biên chỉ để mô phỏng dịch chuyển đầu đọc.

Deadline scheduler đề xuất có thứ tự sector và thứ tự hết hạn: ưu tiên request đã quá hạn; khi chưa có request quá hạn, dispatch theo sector trong batch giới hạn. Công bố thời hạn đọc/ghi, batch limit và tie-break bằng enqueue sequence. Đây là chính sách phần mềm giảm nguy cơ đói tài nguyên; không cam kết mọi request hoàn thành trước hạn thời gian thực.

Chuyển policy lúc chạy thực hiện dưới khóa: giữ id, thời điểm đến và trạng thái request, đổi cấu trúc chọn request còn QUEUED; INFLIGHT tiếp tục hoàn thành bình thường. Reset trạng thái riêng của policy theo quy tắc đã công bố và tăng counter policy_switch. Có thể chọn drain rồi đổi nếu được giảng viên chấp thuận, nhưng phải thể hiện quiesce có kiểm soát, không bỏ request.

Request merging thực chất

Hai request được gộp khi cùng device, cùng operation, liền kề theo sector, cùng miền ordering và không vượt giới hạn kích thước/segment/descriptor. Cần hỗ trợ append và prepend trong test. Đề xuất giai đoạn đầu chỉ merge read; vẫn phải phát một request thiết bị bao phủ nhiều block và trả completion cho từng request con. Chỉ nhóm nhiều phần tử trong một list mà vẫn submit từng lệnh riêng không được tính là device request merging.

Buffer không nhất thiết liền nhau trong RAM. Thiết kế có thể dùng scatter-gather nếu feature/giới hạn cho phép, hoặc bounce buffer có giới hạn. Phải tính cả header và status descriptor, giới hạn queue và NBUF; không chọn queue depth bằng một hằng số tùy ý rồi giả định thiết bị đáp ứng. Sector virtio có đơn vị 512 byte, tổng độ dài đọc/ghi phải đúng bội số và trong capacity. [S7]

Đồng bộ và số đo

Đề xuất một khóa bảo vệ pending queue, request state và kế toán descriptor ở bản đầu; chưa tối ưu nhiều khóa. Ghi rõ lock order, nơi được sleep và quy tắc completion trước khi caller kịp wait. Interrupt không sleep, không cấp phát có thể block, không printf mỗi request trong benchmark. Memory ordering khi publish avail và đọc used cần giữ theo driver/tag đã xác nhận. [S8]

Lưu t_enqueue, t_dispatch và t_complete; từ đó tính queue wait, service và latency toàn phần. Outstanding là số request đã submit cho thiết bị nhưng chưa complete, khác pending trong phần mềm. Counter max_inflight ≥ 2 mới là bằng chứng tối thiểu cho R01.

6 Phân công và cách phối hợp ba người

Người phụ trách	Phần kernel chính	Việc bổ sung	Người review
Hoàng	virtio submit/complete, descriptor lifecycle, error path	Baseline, interrupt trace, tài nguyên	Lợi
Lợi	request queue, FCFS/SCAN/deadline, đổi policy	Trace thuật toán và starvation tests	Hiếu
Hiếu	Async API, buffer lifetime, merging và counters kernel	Test harness, benchmark, báo cáo phép đo	Hoàng

Phân chia này bảo đảm cả ba có phần kernel đáng kể. Hiếu không chỉ viết test và báo cáo. Hoàng cùng Hiếu review mọi thay đổi ownership/merging; Lợi cùng Hoàng review lock và dispatch. Mỗi người viết test cho phần mình, một người khác chạy lại độc lập.

Năng lực thực hiện và ước lượng

Đề xuất tuần thường 12 giờ/người, tổng 36 giờ/nhóm. Tuần 1 dùng 10 giờ/người; tuần 8 dành 14 giờ/người; tuần 9 và 10 dùng lần lượt 10 và 6 giờ/người. Tổng năng lực kế hoạch khoảng 336 giờ. Chỉ cam kết task khoảng 80% năng lực, phần còn lại dành cho tích hợp, review và lỗi phát sinh. Đây là ngân sách thời gian đề xuất, nhóm cần điều chỉnh theo thời khóa biểu thực tế.

Không chia đều số commit làm thước đo duy nhất. Dùng task được nghiệm thu, nội dung PR, code kernel, test và khả năng giải thích để ghi đóng góp. Task lớn hơn 8 giờ nên tách nhỏ; blocker vượt một ngày phải nêu trong nhóm và cập nhật tracker.

Nhịp làm việc mỗi tuần

Đầu tuần họp planning 30 phút để chọn task có owner, reviewer, estimate và DoD. Stand-up hai lần/tuần, mỗi lần 10–15 phút. Cuối tuần review 30 phút, retrospective 15 phút và viết báo cáo dựa trên log/commit. Các giờ họp tính trong ngân sách, không cộng thêm ngầm.

Vai trò PO và Scrum master luân phiên: tuần 1 Hoàng/Lợi, tuần 2 Lợi/Hiếu, tuần 3 Hiếu/Hoàng rồi lặp lại. Người giữ vai trò quản lý vẫn làm kernel. Board gợi ý: Backlog → Ready → In progress → Review → Test → Done; thêm Blocked hoặc nhãn blocker khi cần. Mỗi người tối đa một task lớn ở In progress.

Điều kiện hoàn tất task

Mã build trên commit được review; có assertion/test liên quan và log; reviewer khác tác giả chấp thuận; ghi tác động đến lock, ownership, ordering; cập nhật tài liệu khi interface thay đổi. Done của task khảo sát phải có ghi chú kỹ thuật hoặc sơ đồ và người kiểm tra; không chỉ ghi “đã đọc”. Merge qua PR có issue ID, giữ main chạy được.

Các URL repo, board và design hiện chưa được nhóm cung cấp. Việc tạo/cấp quyền các công cụ này nằm trong checklist tuần 1 và phải được nhóm xác nhận bằng URL thật.

7 Timeline mười tuần và tiêu chí qua mốc

Tuần và khoảng ngày	Công việc trọng tâm	Kết quả và điều kiện qua mốc
1 · 19–25/10	Khởi tạo, đọc code, baseline, kiến trúc	Ba máy boot; có log baseline, board, design và báo cáo thực tế
2 · 26/10–01/11	Pool, submit/wait, completion, FCFS	Read qua đường mới; không mất wakeup; đo được nhiều INFLIGHT
3 · 02–08/11	Chốt interface/lock; SCAN bản đầu	Proposal 2–3 trang; test trace FCFS/SCAN; chốt phạm vi async với GV
4 · 09–15/11	Deadline và chuyển policy	Ba policy có test xác định; đổi lúc có tải không mất request
5 · 16–22/11	Tích hợp core, async demo, merge read	Midterm 10 phút; core hoạt động một phần, test đầu trong CI theo mốc Spec
6 · 23–29/11	Hoàn thiện merge và error path	Checksum, descriptor exhaustion, handle lỗi và tài nguyên đúng
7 · 30/11–06/12	Đo, stress, regression và sửa lỗi	Benchmark ≥5 lần/điểm; stress ≥30 phút; suite CPUS=1/4 ×10
8 · 07–13/12	Buffer sửa lỗi và code freeze	Clean build/test từ clone mới; tag final; đủ log và báo cáo kỹ thuật
9 · 14–20/12	Demo/Q&A, hoàn thiện báo cáo	Demo 20 phút theo Spec nếu lịch giữ nguyên; report 15–20 trang; đóng góp cá nhân
10 · 21–27/12	Đối soát và nộp theo lịch nhóm	Nộp đúng cổng/giờ GV chốt; xác nhận receipt, quyền đọc và tag

Ngày trong bảng là lịch đề xuất theo tuần làm việc, không thay thế thông báo giảng viên. Spec yêu cầu deliverable report ở code freeze và final report ở tuần 9; vì vậy tuần 8 phải có bản kỹ thuật đủ thiết kế, triển khai, kết quả và hạn chế, tuần 9 biên tập bản 15–20 trang. Tuần 10 không dùng để đẩy phần core còn thiếu sang cuối kỳ. [S2]

Đường găng và cách xử lý trễ

Đường găng: môi trường/tag → request lifecycle → FCFS ổn định → ba policy → async/merge tích hợp → test/stress/benchmark → final tag → demo/nộp. Bản thiết kế thuật toán và harness có thể chuẩn bị song song, nhưng chỉ nghiệm thu tích hợp sau khi descriptor/completion đúng.

Nếu tuần 2 chưa có submit/complete ổn định, cả nhóm tập trung driver, dừng stretch. Nếu hết tuần 5 core chưa đủ, bỏ seek-time model/ổ thứ hai và thu hẹp giao diện tùy chọn; vẫn giữ cả ba scheduler, merging và async read là bắt buộc. Nếu tuần 7 còn lỗi dữ liệu/deadlock, ưu tiên sửa correctness, giảm số cấu hình benchmark, giữ số lần lặp tối thiểu.

Sau code freeze, không sửa âm thầm tag final. Nếu được phép hotfix: ghi issue, bằng chứng lỗi, PR review, chạy lại kiểm thử bị ảnh hưởng, xin quy trình chấp nhận tag thay thế và báo giảng viên. Không tự mặc định tuần 10 cho phép thay mã đã freeze.

8 Tuần đầu phải hoàn thành những gì

Tuần 1 thành công khi nhóm có nền tảng thực nghiệm để bắt đầu code tuần 2. Mục tiêu là 30 giờ làm việc của cả nhóm, gồm cả họp và review. Chi tiết theo ngày và số giờ từng người nằm trong 02_Checklist_tuan_1.md.

Ngày	Công việc	Đầu ra kiểm tra được
1 · 19/10	Kickoff, chốt phạm vi và owner; xác nhận lịch/tag/API	Biên bản, bảng câu hỏi, task tuần 1; chưa có câu trả lời thì ghi pending
2 · 20/10	Cài môi trường và clone tag; tạo công cụ nhóm	Version log; ba máy boot; URL repo/Jira/FigJam
3 · 21/10	Đọc đường I/O và khóa, thử baseline	Bản đồ file/hàm; ghi request outstanding và giới hạn descriptor thực tế
4 · 22/10	Thiết kế lifecycle, policy và ownership	Sơ đồ kiến trúc/trạng thái/sequence; quyết định lock và API sơ bộ
5 · 23/10	Baseline regression và chuẩn bị phép đo	Log CPUS=1/4; test matrix; format dữ liệu thô
6 · 24/10	Review thiết kế và chia nhỏ sprint 2	Backlog đủ assignee/estimate/DoD; PR ghi chú; danh sách rủi ro
7 · 25/10	Review, retrospective, báo cáo tuần	Báo cáo thực tế, commit/log/issue links, TODO tuần 2 và check-in 10 phút

Checklist nghiệm thu cuối tuần

1. Cả ba dùng đúng commit gốc, build/boot được; môi trường được mô tả để tái lập.
2. Có repo private, tracker và design; kiểm tra giảng viên đọc được trước mốc tuần học phần 2.
3. Có bản đồ bread/bwrite → driver → interrupt → wakeup, mô tả cả đường descriptor hết.
4. Có baseline stock usertests và grind theo protocol đã ghi; trường hợp lỗi phải có issue và log.
5. Có request lifecycle, ownership, chính sách ordering và quyết định async kernel/user cần chốt.
6. Có backlog toàn dự án; task tuần 2 vừa sức; mỗi người có phần kernel cụ thể.
7. Có kế hoạch ít nhất 30 ca test, ma trận CPU, stress 30 phút và protocol benchmark.
8. Có báo cáo tuần 1 phân biệt Completed/In progress/Blocked, không lấy tỷ lệ trong báo cáo mẫu làm tỷ lệ của nhóm.

Một spike nhỏ có thể thêm counter/log phục vụ khảo sát nếu còn giờ, phải để trên nhánh riêng và review. Không biến spike chưa kiểm chứng thành driver chính thức. Nếu baseline chưa chạy được, ghi là blocker và chuyển giờ còn lại sang sửa môi trường trước khi thêm thuật toán.

Nội dung check-in đầu tiên

Trong 10 phút: 1 phút mục tiêu/phạm vi; 2 phút baseline và môi trường; 3 phút sơ đồ submit/completion cùng rủi ro; 2 phút phân công; 2 phút blocker, câu hỏi và kế hoạch tuần sau. Mỗi người giải thích phần khảo sát của mình và chỉ ra bằng chứng.

9 Kiểm thử và đánh giá hiệu năng

Phân biệt tính đúng và tốc độ

Trước hết chứng minh data integrity, completion đúng một lần, không mất wakeup, không dùng lại buffer sớm và không rò tài nguyên. Sau đó mới so sánh throughput/latency. Một scheduler có thứ tự dispatch đúng vẫn có thể không nhanh hơn trên disk image của QEMU; không đặt giả thuyết “SCAN luôn thắng FCFS” thành tiêu chí PASS.

Kiểm thử chính sách dùng trace đầu vào cố định và oracle độc lập để so thứ tự dispatch, không dùng thứ tự completion thiết bị làm oracle. Trace SCAN phải công bố vị trí đầu đọc, chiều quét, capacity và quy tắc biên. Deadline cần test request quá hạn trong luồng request mới liên tục. Thử chuyển policy khi QUEUED lẫn INFLIGHT khác rỗng.

Protocol benchmark đề xuất

So sánh stock xv6 với bản async FCFS, SCAN và deadline; ở mỗi policy đo merge off/on. Dùng cùng commit, dataset, CPU, seed, queue depth và QEMU command line. Bản stock dùng mức concurrency caller tương đương khi không có API queue depth; ghi rõ phần nào không so ngang trực tiếp. [S2]

Workload tối thiểu: đọc tuần tự, đọc ngẫu nhiên, nhiều reader đồng thời và đọc/ghi hỗn hợp để kiểm tra ảnh hưởng lên đường write. Queue depth thử 1/2/4 hoặc mức thực tế thiết bị/pool cho phép; concurrency riêng ghi 1/2/4. Chuẩn bị file và kiểm tra checksum ngoài đoạn đo, giới hạn kích thước từng file theo xv6 gốc thay vì tự giả định có file hàng chục MB.

Không chỉ lặp đọc một file đã nóng trong buffer cache rồi gọi đó là disk benchmark. Dùng tập nhiều file đủ tạo cache miss hoặc đường benchmark kiểm soát; ghi chế độ cold/warm, đếm I/O thiết bị thực và không sửa trực tiếp metadata filesystem để tạo tải. Mọi direct-block experiment phải chỉ đọc vùng đã chuẩn bị hoặc dùng image thử riêng.

Mỗi điểm ≥5 lần; báo trung bình và độ lệch chuẩn hoặc min–max, kèm p50/p95/p99 từ các sample request đủ lớn. Throughput = byte hoàn tất / thời gian đo; latency toàn phần = complete − enqueue; service = complete − dispatch. Báo cả đơn vị, độ phân giải timer, sample count, lỗi, merge ratio và max_inflight. Timer tick quá thô thì dùng nguồn counter phù hợp và ghi cách quy đổi.

Stress và tiêu chí dừng

Khuyến nghị chạy 30 phút cho từng policy trên CPUS=1 và 4; đây là mở rộng so với yêu cầu đề tài chỉ nêu stress 30 phút. Tối thiểu phải có ca 30 phút bao phủ đường async/merge tích hợp và cả ba policy; ma trận concurrency/stress CPUS=1/4 vẫn phải lặp ≥10 lần theo Spec, có thể dùng các lượt ngắn hơn được ghi riêng. [S1, S2]

Fail nếu checksum sai, panic, timeout không tiến triển, completion trùng/mất, descriptor/request không thu hồi hoặc tài nguyên lệch sau drain và cleanup. Lưu log lỗi thay vì chỉ rerun lấy lần PASS. Kế hoạch chi tiết nằm ở tài liệu 05.

10 Rủi ro và hồ sơ cần nộp

Rủi ro	Mức ưu tiên	Cách giảm và tín hiệu kích hoạt
Mất wakeup hoặc deadlock	Cao	Test complete-before-wait, CPUS=1/4; dump state khi không tiến triển
Use-after-free do DMA hoặc process exit	Cao	Request ownership, drain và handle generation; test kill/exit
Reorder write làm sai logging	Cao	Giữ write synchronous, fence; review trước khi mở rộng
Merge sai dữ liệu hoặc rò descriptor	Cao	Segment bounds, checksum, completion con và exhaustion tests
Không có nhiều request ở queue để lập lịch	Cao	Đo pending/inflight; workload nhiều producer; batching có giới hạn
QEMU không phản ánh seek của HDD	Trung bình	Diễn giải kết quả thật; seek model chỉ stretch, ghi riêng kết quả mô phỏng
Lệch môi trường hoặc tag gốc	Cao	Pin commit/toolchain, tái lập trên máy thứ hai
Một người chỉ làm tài liệu	Cao	Mỗi người một module kernel, PR và Q&A; review chéo
Dồn tích hợp cuối kỳ	Cao	Main chạy được; tuần 5 có core; tuần 7 hoàn tất phép đo

Bộ hồ sơ cuối kỳ

Tại code freeze: source ở tag final; README build một lệnh; test suite một lệnh; user tools; benchmark; log kết quả; bản báo cáo kỹ thuật. Trước demo/nộp: báo cáo 15–20 trang, đồ thị với dữ liệu thô, sơ đồ design, lịch sử progress report, contribution statement từng người và quyền truy cập công cụ còn hiệu lực. [S2]

Demo 20 phút kể cả Q&A: 2 phút kiến trúc; 4 phút async và nhiều outstanding; 4 phút đổi scheduler và trace; 3 phút merging/checksum; 3 phút đồ thị, stress và hạn chế; 4 phút Q&A. Mỗi người trình bày module kernel mình làm. Chuẩn bị workload ngắn tái lập được; log stress 30 phút là bằng chứng bổ sung cho demo trực tiếp.

Nguồn và thứ tự ưu tiên

[S1] Spec/Project catalogue.docx, dòng 26128: yêu cầu riêng, deliverable, demo, stretch và danh sách thành viên.

[S2] Spec/Xv6-riscv Term Projects.docx: quy định, mốc, deliverable, quản lý, rubric và kiểm thử. Lịch nộp tuần 10 do nhóm cung cấp ngày 19/10/2026; các mốc khác cần đối chiếu giảng viên.

[S3] Spec/Progress Report template.docx: cấu trúc và summary 200–300 từ. [S4] Spec/[25224] Progress Report sample week1.docx: chỉ tham khảo trình bày, không dùng kết quả của nhóm khác.

[S5] MIT xv6-riscv, gồm kernel/virtio_disk.c: https://github.com/mit-pdos/xv6-riscv [S6] MIT xv6 book, File system: https://mit-pdos.github.io/xv6-riscv-book/fs.html

[S7] OASIS VIRTIO 1.2, Block Device và Virtqueues: https://docs.oasis-open.org/virtio/virtio/v1.2/virtio-v1.2.html [S8] MIT xv6 book, Locking và Sleep and Wakeup: https://mit-pdos.github.io/xv6-riscv-book/lock.html và https://mit-pdos.github.io/xv6-riscv-book/sleep.html

Tra cứu web ngày 19/10/2026. Khi triển khai, dùng tag và xác nhận của giảng viên làm căn cứ ưu tiên.